

ML-Based Anomaly Detection for Fluid Pipeline Systems

Streamlit Dashboard · Real-Time Simulation · ML Model Integration · Pump & Valve Controls

Domain	Tool	Input	Output
Fluid Pipeline Anomaly Detection	Python + Streamlit + ML Model	Pre-trained model + sensor dataset	Live dashboard with alarms

Problem Statement

I am a PhD Research Scholar in Mechanical Engineering. I have completed an ML-based anomaly detection model for fluid transportation through pipelines. I need help integrating my trained model into a Streamlit dashboard. The dashboard should simulate real-time data streaming from my dataset, display pressure, flow rate, temperature, anomaly status, and leakage alarms, maintain historical trends, and include pump/valve control buttons (simulation only). My ML model is already developed; I only need deployment and dashboard integration.

What this document covers

Part 1	Understanding the Problem	— What pipeline anomaly detection means and why an interactive dashboard is needed
Part 2	Tools and Technologies	— What each library does — !pip install, streamlit, pyngrok, pandas, numpy, matplotlib
Part 3	How All the Pieces Connect	— How the install block, app code, and ngrok tunnel work together
Part 4	Implementation — Full Code	— Every line of your code explained with detailed inline comments
Part 5	Running and Accessing the App	— How to launch in Colab or locally and share the ngrok public URL
Part 6	When Things Go Wrong	— Common errors, their causes, fixes, and the final pre-share checklist

1 UNDERSTANDING THE PROBLEM

What is a fluid pipeline and what can go wrong?

A fluid pipeline is a large network of pipes that carries liquids or gases from one place to another — for example, crude oil across a refinery, or water across a city. Sensors placed along the pipeline constantly measure physical quantities such as pressure, flow rate, and temperature. These readings are logged over time and exported as CSV files for analysis.

Pressure	Flow Rate	Temperature
How hard the fluid pushes against the pipe walls (bar). A sudden drop usually signals a burst or leak.	How fast the fluid moves (m ³ /s). An unexpected drop means fluid is escaping somewhere it should not.	How hot or cold the fluid is. Unusual readings can signal chemical reactions or equipment failure.

What is anomaly detection?

Under normal operation the three sensor readings follow a predictable pattern. An anomaly is any reading that breaks that pattern — suggesting something is going wrong such as a leak, a blockage, or a failing pump. Catching these problems early prevents costly shutdowns, environmental damage, and safety hazards.

How the trained ML model detects anomalies

This dashboard integrates a pre-trained ML model developed as part of a PhD research project in Mechanical Engineering. The model was trained on labelled sensor data from fluid pipeline systems and classifies each time-step as normal or anomalous. Rather than applying a simple statistical threshold, the model captures complex non-linear patterns in pressure, flow rate, and temperature to flag genuine faults — leaks, blockages, or pump failures — with higher precision than rule-based methods.

Why a browser dashboard instead of a notebook?

A Jupyter notebook only works while it is open on the analyst's computer. Streamlit turns the Python script into a proper web app that anyone can use in a browser — no Python knowledge needed. pyngrok then creates a secure public tunnel so the local Streamlit server is accessible from any device anywhere in the world using a free ngrok URL.

Part 2 TOOLS AND TECHNOLOGIES

Before looking at any code it is worth understanding what each tool is and why it is used. You do not need to be an expert in any of them — just know what each one does.

!pip install	<pre>streamlit pyngrok pandas numpy matplotlib scikit-learn</pre> Jupyter magic command that installs Python packages from PyPI. The ! prefix tells Jupyter to run the line as a shell command rather than Python. Each package name after install is downloaded and made available to import in any cell.
Streamlit	<pre>import streamlit as st</pre> A free Python library that turns a normal Python script into an interactive web app with no HTML, CSS, or JavaScript needed. <code>st.title()</code>, <code>st.file_uploader()</code>, <code>st.selectbox()</code>, <code>st.line_chart()</code>, and <code>st.write()</code> are all Streamlit functions used directly in your code.
pyngrok	<pre>from pyngrok import ngrok</pre> A Python wrapper around the ngrok command-line tool. ngrok opens a secure public HTTPS URL that tunnels traffic to your local Streamlit server on port 8501. Anyone with the URL can use your app without needing Python installed. You must set your auth token before creating
pandas	<pre>import pandas as pd</pre> The standard Python library for tabular data. <code>pd.read_csv()</code> reads the uploaded CSV into a DataFrame. <code>df.columns</code> gives the column names for the dropdown. <code>df[col].mean()</code> and <code>df[col].std()</code> compute the statistics for anomaly detection.
NumPy	<pre>import numpy as np</pre> A library for numerical computation. Although not called directly in your visible code, NumPy powers pandas internals. It is imported as standard practice alongside pandas for any numerical pipeline work or future model integration.
matplotlib	<pre>import matplotlib.pyplot as plt</pre> Python's foundational plotting library. <code>st.line_chart()</code> can accept matplotlib figures. It is imported here as a standard companion to pandas so you can add custom plots (histograms, scatter plots) without installing anything extra.
scikit-learn	<pre>from sklearn import ... (future use)</pre> The most popular Python machine learning library. It is installed now so that you can later replace the $\text{mean} \pm 2\sigma$ rule with a trained model such as IsolationForest or RandomForestClassifier without needing to install anything extra later.

3 HOW ALL THE PIECES CONNECT

The four code blocks and what each one does

Your entire implementation is a single notebook or script with four logical blocks that run in sequence. There is no separate folder structure — everything is self-contained.

1 Block 1 — Install libraries

The `!pip install` line runs first. It downloads and installs all six libraries from PyPI into the current Python environment. This only needs to run once — after that the libraries are cached and the line can be skipped on future runs.

2 Block 2 — Write the Streamlit app (`%%writefile app.py`)

The `%%writefile` magic command saves everything in the cell into a file called `app.py` on disk. The file contains the full Streamlit application: page title, file uploader, column selector, line chart, and anomaly detection table. This file is what Streamlit will run in Block 3.

3 Block 3 — Launch Streamlit

`!streamlit run app.py &>/dev/null &` starts the Streamlit server as a background process. The `&>/dev/null` part suppresses console output so your notebook stays clean. `time.sleep(5)` gives Streamlit five seconds to fully start up before Block 4 runs.

4 Block 4 — Create the ngrok tunnel

`ngrok.set_auth_token()` authenticates your ngrok account. `ngrok.connect(8501)` opens a secure public HTTPS tunnel to `localhost:8501` — the port Streamlit listens on. `print(public_url)` outputs the clickable link that anyone can open in a browser.

What happens when a user visits the app

Step 1 · Open the URL Their browser connects to the ngrok server, which forwards to Streamlit on port 8501.	Step 2 · Upload a CSV <code>st.file_uploader()</code> receives the file. <code>pd.read_csv()</code> parses it into a DataFrame. <code>df.head()</code> shows the first 5 rows.
Step 3 · Select a column <code>st.selectbox()</code> shows a dropdown of every column name. The user picks the sensor to analyse.	Step 4 · Line chart <code>st.line_chart(df[col])</code> draws an interactive time-series plot of the selected column.
Step 5 · Anomaly table mean and std are computed. Rows outside $\pm 2 \times \text{std}$ are collected and displayed as a table.	Step 6 · Iterate The user can change the dropdown to switch columns and re-run the analysis instantly.

Part 4 IMPLEMENTATION — ALL THE CODE

1 Install all required libraries

Run this cell first. It installs six Python packages from PyPI. The `!` prefix tells Jupyter to execute it as a shell command rather than Python code.

```
!pip install streamlit pyngrok pandas numpy matplotlib scikit-learn

# streamlit    - turns a Python script into a browser web app
# pyngrok      - creates a public URL tunnel to your local server
# pandas       - reads CSV files and computes statistics
# numpy        - numerical array operations (used by pandas internally)
# matplotlib   - plotting library (imported as standard companion)
# scikit-learn - ML algorithms ready for future model integration
```

Note:

After the first install the packages are cached — you can comment this line out on later runs.

2 Write the Streamlit app to disk (%%writefile app.py)

`%%writefile` is a Jupyter cell magic. Everything in the cell below it is written to the file `app.py` on disk — it is NOT executed as Python yet. Streamlit will run this file in Block 3. Every line is annotated in the comments below.

```
%%writefile app.py

import streamlit as st    # web app framework
import pandas as pd       # CSV reading + statistics
import numpy as np        # numerical operations
import matplotlib.pyplot as plt # available for custom charts

st.title('Pipeline Anomaly Detection')
# Displays the main heading at the top of the browser page.

file = st.file_uploader('Upload CSV', type=['csv'])
# Shows a file upload button that accepts only .csv files.
# When the user picks a file, 'file' holds the uploaded data object.
# Until a file is uploaded, 'file' is None and the block below is skipped.

if file:
    df = pd.read_csv(file)
    # pd.read_csv() parses the CSV into a DataFrame —
    # rows = time steps, columns = sensor readings.

    st.write(df.head())
    # Shows the first 5 rows so the user can verify the data loaded correctly.

    col = st.selectbox('Select column', df.columns)
    # Renders a dropdown menu of every column in the CSV.
    # The user picks the sensor to analyse (e.g. 'pressure', 'flow_rate').

    st.line_chart(df[col])
    # Draws an interactive line chart of all values in the selected column.
    # x-axis = row index (time step); y-axis = sensor value.

    mean = df[col].mean() # arithmetic average of the selected column
    std  = df[col].std()  # standard deviation — how spread out values are

    anomalies = df[(df[col] > mean + 2*std) | (df[col] < mean - 2*std)]
    # Statistical anomaly rule: any row whose value is more than 2 standard
    # deviations from the mean is flagged. The | operator means OR.
    # ~95% of normal readings fall within mean ± 2*std, so anything outside
    # that band is statistically unusual and worth investigating.
```

Part 4 IMPLEMENTATION — CONTINUED

3 Launch the Streamlit server

This block starts the Streamlit app as a background process and waits five seconds for it to fully start before the tunnel is created in Block 4.

```
!streamlit run app.py &>/dev/null &
# !                - run as a shell command, not Python
# streamlit run app.py - start the Streamlit server for app.py
# &>/dev/null      - redirect all output so the notebook stays clean
# &               - run the process in the background; do not wait

import time
time.sleep(5)
# Pause 5 seconds to give Streamlit time to start up.
# Without this the tunnel in Block 4 may connect before Streamlit is ready.
```

Important:

Streamlit runs on port 8501 by default. If another Streamlit process is already running on that port, this block will fail. Kill old processes with: `!kill -f streamlit`

4 Create the ngrok public tunnel

This block authenticates your ngrok account and opens a secure public HTTPS tunnel from the internet to your local Streamlit server. The printed URL is what you — or anyone else — opens in a browser to access the app.

```
from pyngrok import ngrok
# Import the ngrok Python wrapper.

ngrok.set_auth_token("YOUR_NGROK_AUTH_TOKEN_HERE")
# Authenticates your ngrok account using your personal auth token.
# Without this, ngrok will only work briefly before asking you to sign up.
# Get your token from: dashboard.ngrok.com/auth
# SECURITY: Replace the token shown above with your own.

public_url = ngrok.connect(8501)
# Opens a tunnel from the public internet to localhost:8501.
# ngrok assigns a random HTTPS URL such as https://abc123.ngrok.io
# that forwards all traffic to your local Streamlit server.

print(public_url)
# Prints the NgrokTunnel object. The URL is visible in the output.
# Example: NgrokTunnel: 'https://abc123.ngrok.io' -> 'http://localhost:8501'
# Copy and paste that URL into any browser to view your dashboard.
```

Security reminder:

Your auth token is a personal credential — anyone who has it can open tunnels on your account. Never commit it to GitHub or share it publicly. Replace it with your own token.

Part 5 RUNNING AND ACCESSING THE APP

Running in Google Colab (recommended)

Google Colab is the ideal environment because it already has Python and Jupyter in a cloud machine. You do not need to install anything on your own computer.

- 1 Open Google Colab**
Go to colab.research.google.com and create a new notebook.
- 2 Run Block 1**
Paste the `!pip install` line into a cell and run it. Wait for all packages to install.
- 3 Run Block 2**
Paste the `%%writefile` block into a cell and run it. The file `app.py` is created on disk.
- 4 Run Block 3**
Paste the `!streamlit run` block into a cell and run it. The server starts in the background.
- 5 Run Block 4**
Paste the `pyngrok` block into a cell and run it.
- 6 Open the URL**
Copy the printed `ngrok` URL and open it in any browser tab.

Running locally (terminal)

You can also run the code from a local terminal if Python is installed.

```
# Step 1 – install libraries
pip install streamlit pyngrok pandas numpy matplotlib scikit-learn

# Step 2 – save your app code to app.py
#         (copy the content of Block 2, minus the %%writefile line)

# Step 3 – run Streamlit
streamlit run app.py

# Step 4 – in a separate Python session, create the tunnel
from pyngrok import ngrok
ngrok.set_auth_token("YOUR_TOKEN_HERE")
public_url = ngrok.connect(8501)
print(public_url)
```

Using the app once it is running

- | | |
|------------------------|---|
| Open the URL | Click or paste the <code>ngrok</code> URL into any browser. The Streamlit page loads instantly. |
| Upload a CSV | Click 'Browse files' and pick your sensor CSV. The first 5 rows appear to verify the data. |
| Select a column | Use the dropdown to choose the sensor column you want to analyse (e.g. pressure). |
| Read the chart | The line chart shows all readings over time. Look for sharp spikes or dips. |
| Check the table | Scroll down to see the anomalies table — each row is a statistically flagged point. |
| Switch columns | Change the dropdown selection to analyse a different sensor column instantly. |

6 WHEN THINGS GO WRONG

Error / Symptom	What it means	How to fix it
ModuleNotFoundError	Python cannot find a library you imported.	Re-run the !pip install block with the virtual environment active.
'NoneType' has no .read() / app crashes on upload	The if file: guard was removed or skipped.	Wrap all CSV code inside if file: so it only runs after upload.
KeyError: 'column_name'	That column does not exist in the uploaded CSV.	Print df.columns to screen to see the actual header names.
st.line_chart raises error	Selected column has non-numeric values.	Add: <code>df[col] = pd.to_numeric(df[col], errors='coerce')</code>
Empty or too many anomalies	2×std threshold may be too tight or too loose.	Try 1.5×std for more results or 3×std for fewer.
ngrok URL loads blank page	Streamlit has not finished starting when ngrok connected.	Increase <code>time.sleep(5)</code> to <code>time.sleep(10)</code> and rerun.
ngrok auth token error	Token is invalid, expired, or not yet set.	Get a fresh token at dashboard.ngrok.com/auth .
App freezes on second run	A second Streamlit process started on port 8501.	Run <code>!kill -f streamlit</code> then re-run blocks 3 and 4.

Final Checklist

Verify each item before sharing the URL with anyone

- Block 1 ran successfully — no red errors and all six libraries installed
- Block 2 ran and printed 'Writing app.py' — the file exists on disk
- Block 3 ran — Streamlit server started in the background on port 8501
- `time.sleep(5)` completed before Block 4 ran — Streamlit was given time to start
- Your ngrok auth token is set correctly and is your own personal token
- Block 4 printed a URL starting with `https://` — the tunnel is live
- You opened the URL in a browser and saw the 'Pipeline Anomaly Detection' title
- You uploaded a CSV and confirmed the line chart and anomaly table both appear
- Any text/non-numeric columns are handled with `pd.to_numeric()` or excluded